

agent-runtime

Brief: an event-sourced runtime for tool-using LLM agents

A concise, accurate overview

Scope of this document

A condensed but accurate account of **agent-runtime**: what it is, the one idea it is built on, what that idea buys, and where it leaks. Claims that could be checked by running the code were checked; the transcripts here are real output. Terms are defined in the glossary at the end rather than inline.

For the full treatment, every module and every function of consequence, see the companion **deep-dive.pdf** in this folder. This is not an excerpt of it: it is the same project from higher up.

Contents

1	The problem it picks	2
2	Architecture	3
3	The event log	3
4	What the log buys	4
4.1	The approval gate	4
4.2	Crash safety	4
4.3	Deterministic replay	5
4.4	Failure injection	5
4.5	Regression testing between model versions	5
5	Tools, adapters, agents	6
6	Verification: what was actually run	6
7	Honest findings	7
8	Glossary	9

1 The problem it picks

Writing an agent loop is easy. A model is asked what to do, it names a tool, the program runs the tool, the result goes back to the model, repeat until it answers. That is a weekend.

What is hard is everything that makes such a thing safe to deploy:

- What exactly happened in run 4187, and can I reproduce it?
- Did the agent email a customer before anyone approved it?
- If I upgrade the model, do the 24 flows that worked last week still work?
- It crashed halfway. Can I resume without sending that email twice?

None of those are questions about the loop. They are questions about the record. This project's bet is that if you get the record right, all four stop being separate features.

The single design decision

Every run is an **append-only log of events** in SQLite. The full state of a run (status, conversation, pending approvals, executed tools, cost) is a pure function of that log, computed on demand. Nothing else is authoritative: no session object survives a process, and none needs to.

Everything else in this document is a consequence of that sentence.

Storing the log instead of the current state costs disk and a little speed. In return it answers questions that overwriting cannot answer at all: what was true at step 6, why did it do that, do it again exactly. For a component that is nondeterministic, expensive per step, and acts irreversibly on the world, that trade is heavily favorable. An agent is exactly such a component.

2 Architecture

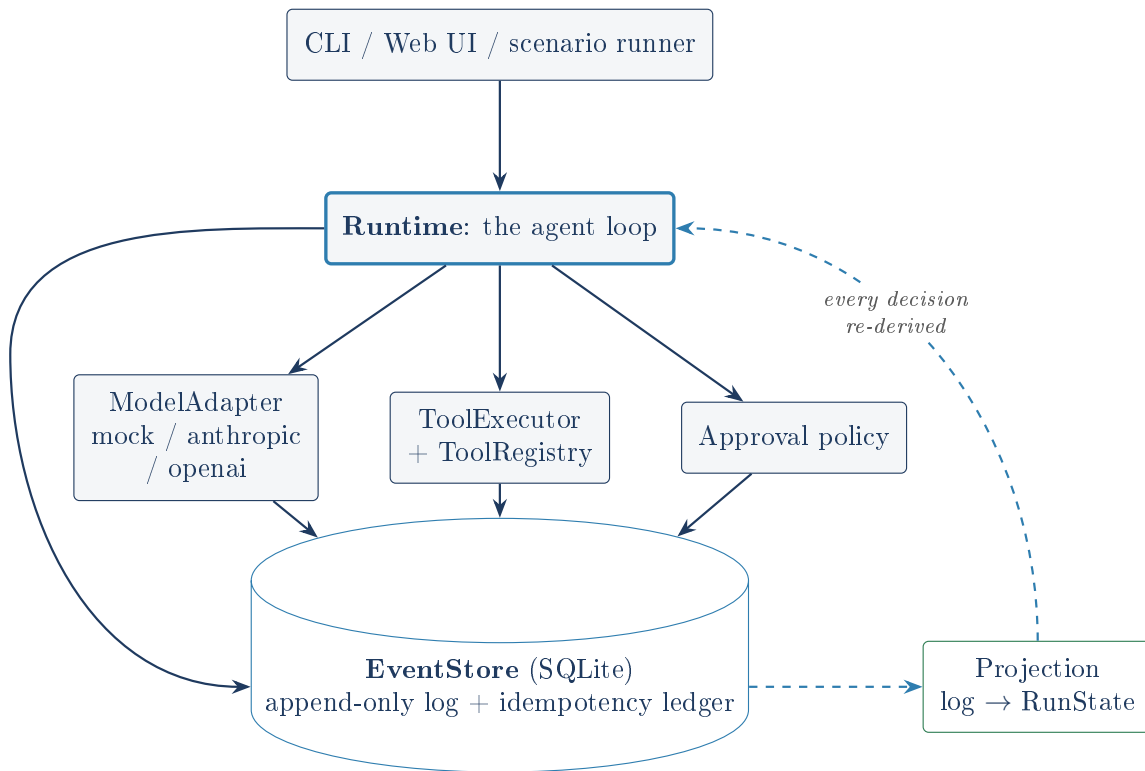


Figure 1: Solid arrows append; dashed arrows read through the projection. No SQL statement in `store.py` updates or deletes an event row.

`projection.py` is the single module that turns a log into meaning. The CLI, the web UI, the replay comparator and the scenario assertions all consume its `RunState`, so they cannot disagree about a run’s status. This is enforced by structure rather than convention: `events.py`, `store.py` and `projection.py` sit at the bottom of the dependency graph and import nothing from the rest of the package, so no other module is even positioned to reinterpret a log.

The loop itself, `Runtime.drive()`, repeats four steps: project the log; if terminal, stop; resolve pending tool calls in order (read-only ones execute, side-effecting ones need an approval decision or the run pauses); otherwise ask the model for the next turn and append what happened.

Why this collapses three features into one

Because step one re-derives everything from the log, “resume after a crash”, “continue after a human approves” and “take the next ordinary step” are not three code paths. They are one. There is no recovery logic anywhere in the codebase, because there is nothing to recover: the loop cannot tell whether the last event was written one millisecond or one week ago. That is the practical payoff of the whole design, and the reason the recovery tests exercise the normal path instead of a bespoke one.

3 The event log

One table, `events(run_id, seq, type, ts, payload)`, primary key `(run_id, seq)`. Twelve event types cover a run’s whole life: `run_created`, `model_requested`, `model_responded`, `model_error`, `tool_requested`, `tool_approved`, `tool_denied`, `tool_executed`, `tool_failed`, `checkpoint`, `run_finished` and `run_failed`.

Appends compute $\text{MAX}(\text{seq})+1$ and retry on the uniqueness violation, so two writers racing on one run cannot both claim a sequence number, with no lock involved. Every event has a *canonical* JSON encoding (sorted keys, no whitespace) so two logs can be compared for exact equality. That is what lets replay say “byte-identical” rather than “looks about right”.

A real 15-event run, produced by actually running the shipped demo scenario:



Figure 2: The gate is the only place a run waits, and it waits by stopping, not by blocking.

4 What the log buys

4.1 The approval gate

Side-effecting tools (write a file, update a CSV, draft an email, add a calendar entry) stop the run. An *approval policy* is just a function from a call to `approve`, `deny(reason)`, or `None` meaning “ask a human”.

The important part is what a pending approval *is*: not a blocked thread and not an in-memory queue, but a fact about the log, namely a `tool_requested` event with `side_effect: true` and no decision event after it. The process can exit. Any process, days later, can read the log and see it. That is why the CLI and the web UI serve the same queue while sharing nothing but a file. A denial is fed back to the model as an ordinary observation (`{"denied": true, "reason": ...}`), so “the operator said no” is something the agent can react to rather than an error.

4.2 Crash safety

The executor derives an idempotency key from `(run_id, seq of the tool_requested event)`. Both are identical on resume, so the key is stable across the crash it is meant to survive. It writes the result to an `INSERT OR IGNORE` ledger *before* the runtime appends `tool_executed`; a re-

sumed run finds the ledger entry and reports `replayed: true` instead of re-running the handler. `tests/test_resume.py` crashes a run in that window and asserts a side-effect counter stays at 1.

4.3 Deterministic replay

`agentrun replay <run-id>` rebuilds a run from four recorded stand-ins: a clock replaying the original timestamps in append order, an adapter replaying recorded turns and errors, an executor replaying recorded tool outcomes, and an approval policy replaying recorded decisions. The *real*, *unmodified* runtime then drives the loop. Nothing is swapped except the run’s inputs, which is the point: a loop reimplemented for replay would prove nothing about the loop.

Why replay matters more for an agent than for ordinary software

Suppose a run drafted an email to the wrong customer and you want to know why. You cannot just run it again: the model is nondeterministic, so a second run may behave perfectly and tell you nothing; it costs money every attempt; and it would draft another email, so your debugging tool has side effects.

Replay dissolves all three. The log already holds every nondeterministic input the run received. Feed those back and the run becomes a pure function: same events every time, free, no email sent, steppable, a year later.

This works only because every event payload was designed so replay can recompute it. The store takes an injectable clock, sleeps become no-ops, and `model_requested` stores a digest rather than anything time-dependent. Determinism was spent field by field, which is what lets the comparator demand exact equality instead of “close enough”.

Verified, not quoted:

```
1 $ agentrun replay 46bd97a45451
2 replayed 15/15 events into 46bd97a45451-replay-8765c6
3 byte-identical: yes
```

4.4 Failure injection

A YAML-declarable `FaultPlan` names faults at points: model faults per adapter call (`malformed`, `adapter_500`) and tool faults per attempt (`timeout`, `exception`). The contract: adapter errors retry with backoff then fail the run with a recorded cause; malformed output earns two corrective observations then fails; tool failures surface to the model as ordinary error observations. Nothing escapes `drive()` as an unhandled exception.

4.5 Regression testing between model versions

What is actually being tested

You cannot unit-test “the agent does the right thing”, because the agent’s behavior lives in a model you do not control. So the project tests the loop against a *recorded* behavior: a scenario names behavior versions, each a full script of what the model would say, and asserts what the runtime does with it.

That turns an unanswerable question into an answerable one: not “is the agent correct” but “if the model behaves like this, does the runtime do the right thing, and does v2 differ from v1, and where exactly”. Offline, in seconds, for free.

24 YAML scenarios ship, 8 per agent. Each has a request, optional fixture files, named behaviors, optional faults and approvals policy, and an `expect` block: exact or partial tool lists, denied tools, final-answer substrings, failure cause, files created, minimum event count, and a structural check that no side effect executed before its approval. Verified:

```

1 $ agentrun test scenarios/ --behavior v1
2 behavior v1: 24/24 scenarios passed
3
4 $ agentrun test scenarios/ --behavior v1 --compare v2
5 comparing v1 -> v2: v2 fails 3 of 24 scenarios (3 regressions vs v1)
6 [REGRESSION] chase-overdue-acme
7     first divergence at event 3: expected tool_requested:read_file,
8     got tool_requested:draft_email

```

The v2 behaviors are deliberately degraded: this one drafts the email without reading the invoice first. The mechanism is pointedly simple. Each event is reduced to a "type:detail" string and two lists are walked in lock-step to the first mismatch. No diffing library, because for an ordered list of short strings, first-mismatch is the complete answer. That single line is the most useful output the tool produces.

5 Tools, adapters, agents

Each tool declares a JSON-schema for its arguments, a `side_effect` flag, a timeout and a retry budget. The executor validates arguments, runs the handler under a timeout, and splits failures two ways: `ToolError` is an expected deterministic refusal (sandbox escape, allowlist miss, missing file) and fails fast, since retrying cannot help; anything else is transient and consumes the retry budget with backoff. That one distinction buys the whole retry policy. All file paths resolve through a workspace sandbox that rejects escapes.

Tool	What it does	Side effect
<code>read_file</code>	read a UTF-8 text file	no
<code>write_file</code>	write a UTF-8 text file	yes
<code>list_files</code>	recursively list files	no
<code>http_get</code>	GET an allowlisted URL	no
<code>csv_update</code>	append a row or set a cell in a CSV	yes
<code>draft_email</code>	write an .eml draft file; never sends	yes
<code>calendar_add</code>	insert into a per-workspace SQLite calendar	yes
<code>calendar_list</code>	read the calendar back	no

One interface, `ModelAdapter.complete(messages, tools) → ModelTurn`, has three implementations: `MockModelAdapter` (a deterministic offline script player), `AnthropicAdapter` (Messages API) and `OpenAIAdapter` (Chat Completions), both env-key-driven over `httpx` and both accepting an injectable client, which is what makes them testable with no network.

Three example agents each pair a system prompt with a restricted tool subset, enforced by the registry rather than by the prompt: `ops_assistant` (chases overdue invoices, drafts reminders, never sends), `data_entry` (transcribes to CSV, verifies by reading back), `research_summarizer` (fetches allowlisted pages and notes, writes a sourced summary).

The web UI deserves one line beyond its existence: pending approvals are rendered as a real unified diff of what the file would become, not as a blob of JSON arguments. An approval gate showing JSON gets rubber-stamped within a day. One showing the actual diff is a control someone might really read.

6 Verification: what was actually run

The README claims “93 tests, all offline” and that no live API calls were made. Both were checked rather than trusted.

```
1 $ pytest -q
2 93 passed, 1 warning in 46.80s
```

13 test files, 1357 lines, 93 functions matching `^def test_`, and `pytest --collect-only` reports 93 collected. There is no `parametrize` anywhere in `tests/`, so the count is not inflated: the README's number is exact. The single warning comes from FastAPI's `TestClient`, inside the library, not from this project.

A passing offline suite does not by itself prove nothing called out: a call could succeed silently or be swallowed by a retry. So the suite was re-run with a guard patching `socket.socket.connect`, `socket.create_connection` and `socket.getaddrinfo` to raise on any non-loopback address:

```
1 $ pytest -q -p noweb
2 NETWORK GUARD: zero outbound connections or DNS lookups attempted.
3 93 passed, 1 warning in 55.01s
```

Not one outbound connection, and not even a DNS lookup. The claim is stronger than the README states it: the suite does not merely avoid calling the providers, it never resolves their hostnames.

Beyond the suite, the CLI was driven end to end: a run pausing at the gate, a CLI approval resuming it to `finished`, a full replay reporting `byte-identical: yes`, a `--until` replay reporting mid-run state, 24/24 scenarios passing on `v1`, and the `v1` against `v2` diff reporting 3 regressions with first-divergence event numbers. Every transcript in this document is real output.

Why the offline property is architecture, not luck

Nothing is skipped for lack of a key and nothing is patched at the last moment. The adapters take an injectable client, the runtime an injectable adapter, the store an injectable clock, the executor an injectable sleep and fault hook. Offline falls out of the constructor signatures. That is also its honest limit: the same design that runs 93 tests with no network is why none of them can tell you the live adapters work.

7 Honest findings

Running and reading the code surfaced defects the README does not mention. Each is reproduced and evidenced in the deep dive at the section given.

A truncated replay can execute a real side effect (Deep Dive Sec. 9.4)

Replay writes a full duplicate run into the *same* database, copying the original's metadata verbatim, marked as a replay only by a substring in its run id that no other module checks. Replay with `--until` truncated at a pending side-effecting call and the replayed run stops at `waiting_approval`, leaving a phantom entry in the real approval queue:

```
1 $ agentrun approvals list
2 d5ca78675af3-replay-6c2258 call-2-0 draft_email {'to': 'billing@acme.example', ...}
```

Approving it rebuilds a runtime from that copied metadata, with a *real* executor over the *real* workspace, and the tool fires for real. Confirmed by deleting the outbox first, so a reappearing file could only mean the handler genuinely ran; it reappeared. The idempotency ledger cannot help, since its key includes the run id and the replay's is different.

"Replay: no network, no tool handlers" is true *during* replay. It is not true afterwards, because a truncated replay leaves behind a live, resumable run that no longer knows it was ever a replay. Either mark replayed runs in their payload and have resume and the queue refuse them, or replay into a throwaway database. The second is cleaner: a replay is a question, not a fact, and has no business being stored beside real runs.

Resume rewinds the script after a scripted malformed turn (Deep Dive Sec. 18)

The mock adapter advances its script index before inspecting the turn, so a *scripted* malformed turn consumes a script slot. But `resume_run` repositions that index by counting `model_responded` events that are **not** malformed, so it does not count that turn. Resume therefore rewinds by one and serves the malformed turn again, which can fail a run that would otherwise have completed.

The README concedes a nearby case (injected faults realigning off by one) but not this one. Two shipped scenarios use scripted malformed turns; neither is ever resumed by the tests, which is why 93 tests pass over a real bug.

The generalization is the interesting part. The project's thesis is "all state is a function of the log". The script cursor is the one piece of state that is not, and it is inferred by a heuristic rather than recorded. Every feature that reads the log is exact; the one component that guesses is the one with a latent bug. That is an argument *for* the discipline, not against it. The README already names the fix: record the cursor in the `checkpoint` payload, which is currently carrying only numbers that were recomputable anyway.

The crash window is narrower than claimed, not closed (Deep Dive Sec. 8)

The ledger write happens on the line after the handler returns. A crash *between* those two statements leaves the side effect done and nothing recorded, and a resumed run runs the handler again. The test does not cover it: `CrashAfterExecute` raises after calling the inner executor *in full*, ledger write included, so it crashes in the window that is closed. Its docstring calls it "the worst possible crash point", which is the one thing it is not.

This is defensible and worth defending precisely. The gap is inherent: you cannot atomically perform a real-world side effect and a local database write. Shrinking the window is the best a local ledger can do; closing it requires the side effect to carry the idempotency key so the *remote* system deduplicates, as payment APIs do. The honest claim is "the duplicate window is reduced to two adjacent statements", not "no duplicate side effects".

Also found, and detailed in the deep dive: the web approval gate has no authentication or CSRF protection (mitigated by the localhost default bind, but the gate *is* the security control this project is built around); the approval preview re-implements tool logic rather than sharing it and already disagrees with the real tool on out-of-range cells; `agentrun.db` is committed to the repository and the default database path writes into the working tree; `except RuntimeError: pass` in replay swallows genuine store and adapter errors and mislabels them as divergence; a too-short mock script is reported as malformed *model* output rather than as the scenario bug it is; `draft_email` silently overwrites drafts sharing a subject; and a hung tool does not merely leak a thread, it hangs `execute()` itself past the timeout.

The README states these on its own: live adapters were never exercised against the real APIs (no keys were available); web approvals resume synchronously in the request handler; tool timeouts abandon rather than kill the worker thread; event payloads carry no schema version; the price table is hardcoded and will drift.

The fair summary

Every finding above is a defect in the operational shell. Not one is a defect in the central idea. The event log, the projection as sole interpreter, the gate as an ordering property rather than a lock, and byte-identical replay all hold up under examination and under execution: 93 tests pass with provably zero network, replay is byte-identical on real runs, and the regression diff does what it claims.

What the findings show is narrower and more interesting: the seams. The phantom approval

happens because a replay is written into the same table as a real run and distinguished only by a naming convention. The resume bug happens because one piece of state was inferred rather than recorded. Both are the project not taking its own discipline quite far enough, which is a far better thing to defend than a design that does not work.

Of the limitations the README raises itself, the unversioned event payload is the one most worth fixing first. For a project whose whole premise is that the log is the durable asset, a log that cannot say which code wrote it is the one structural gap that is cheap now and expensive later.

8 Glossary

Adapter	A class implementing a fixed interface around a specific model API or a fake.
Approval gate	The point where a side-effecting call pauses for a human decision.
Canonical encoding	A serialization where equal data always produces an identical string.
Contract test	A test verifying request and response shape against an API's spec without calling it.
Deterministic	Same inputs, same outputs, every time.
Event sourcing	Storing every state change as an immutable ordered log, and computing current state from it on demand, instead of overwriting stored state.
Fault injection	Deliberately simulating failures to test that they are handled.
Idempotent	Doing something twice has the same effect as doing it once.
JSON Schema	A specification language describing the required shape of JSON data.
LLM	Large language model: a model that generates text or structured output from a prompt.
Projection	Computing derived state from an event log.
Replay	Re-driving logic using only recorded data, with no live calls, checking the result matches.
Sandbox	A restricted environment limiting what a program can access.
Side effect	An action with a lasting, observable consequence outside the program.
SQLite	A serverless relational database engine whose whole database is one file.
Unified diff	A compact standard format for showing line-level differences between two texts.